

JobXApply — Resume & LinkedIn Import for Onboarding

Skip manual data entry: parse an uploaded resume or an official LinkedIn export directly into the Profile

Right now, onboarding asks a new user to manually type everything into the Profile Setup flow. This document specifies letting them instead upload an existing resume (from their device or Google Drive) or import from LinkedIn, with the system parsing that source directly into Profile Schema fields -- always ending in a review screen the user confirms before anything is saved, never auto-committing parsed data silently.

LinkedIn import needed real research before designing this -- LinkedIn's official API does not support third-party profile import for a consumer product like this. Recruiting/talent-related data extraction is explicitly listed as a restricted use case in LinkedIn's own API terms, and scraping (official or unofficial endpoints) carries real enforcement risk, including account bans, evaluated by LinkedIn based on session behavior. Section 3 below routes around this using LinkedIn's own official self-service export features instead, which is both safer and requires no partner approval.

1. Resume Upload & Parsing Pipeline

Upload sources offered at onboarding:

- **Local device** -- standard file picker, accepts PDF and DOCX.
- **Google Drive** -- Google Drive Picker API (official, OAuth-based, well-supported) lets the user browse and select a file from their Drive without granting the app broad Drive access -- the Picker only returns the specific file(s) the user selects, not a full Drive listing.
- **Other cloud sources (later phase)** -- OneDrive/Dropbox pickers follow the same pattern if there's user demand; not required for v1.

Parsing pipeline (reuses the ATS Checker's PDF analysis engine):

1. **UPLOAD** -> file received (local picker or Drive Picker returns a file reference)
2. **EXTRACT** -> PDF: pdf.js-style text + position extraction (same layout-analysis approach as the ATS Score Checker); DOCX: mammoth-style text extraction
3. **SEGMENT** -> split raw text into candidate sections (Summary, Experience, Education, Skills, Projects, Certifications) using header-pattern matching
4. **FIELD-PARSE** -> within each section, extract structured fields:
 - Experience: company, title, date range, bullet points
 - Education: institution, degree, date range, grade
 - Skills: flat list, de-duplicated
 - Contact: email/phone via regex, LinkedIn/GitHub URLs via pattern match
5. **CONFIDENCE** -> score each extracted field's confidence (same tiering logic as the autofill engine: high/medium/low) based on how cleanly it matched an expected pattern vs. how much guessing was required
6. **MAP** -> map extracted fields onto Profile Schema fields exactly (personal.*, education[], experience[], skills[], projects[], certifications[])
7. **REVIEW** -> present a side-by-side review screen (Section 4) -- nothing is saved

to the Profile until the user confirms it, field by field if needed
8. COMMIT -> only on explicit user confirmation, write to Profile via the normal
Profile-editor write path (per the Profile Schema's read-only-consumer
architecture rule -- the parser is not a special-cased writer)

Reusing the ATS Checker's PDF layout-analysis code here is a deliberate efficiency: the same real text-and-position extraction that detects multi-column layouts and embedded images for scoring purposes is exactly what's needed to reliably pull structured fields out of a resume. Two features, one extraction engine.

2. Extracted Field -> Profile Schema Mapping

Detected in resume	Maps to	Typical confidence
Name (usually the largest text on the page, top of document)	personal.fullName	High if isolated as the top line in large font; medium if inferred from email address.
Email / phone	personal.email / personal.phone	High -- regex-matched, low ambiguity.
LinkedIn / GitHub / portfolio URLs	personal.links[]	High if a full URL is present; medium if only a handle/username is found near a recognizable label.
Address / city	personal.address.city, state	Medium -- free text, no strict pattern; often absent entirely on modern resumes.
Each Experience entry	experience[] (jobTitle, company, dates, description)	High for company/title if clearly formatted; description bullets carry over verbatim -- no rewriting during import.
Each Education entry	education[] (degree, institution, dates, grade)	High if a recognizable degree pattern (B.Tech, B.Sc, M.Tech, etc.) is present; medium otherwise.
Skills list	skills[]	High for a clearly labeled Skills section; lower confidence for skills only inferred from experience bullet text -- those are surfaced as suggestions, not auto-added.
Projects	projects[]	Medium -- project sections vary the most in format across resumes.
Certifications	certifications[]	High if a labeled Certifications section exists; otherwise not populated rather than guessed.

Hard rule: a field with low confidence is shown to the user as a suggestion to confirm or correct, never silently written. This mirrors the exact confidence-threshold philosophy already established for the autofill engine -- the parser should be honest about uncertainty rather than presenting a guess as a fact.

3. LinkedIn Import -- Official Routes Only

LinkedIn's Profile API via OAuth grants login and the user's own basic profile only -- it does not expose full work history, education, or skills for import into a third-party app, and using it (or any unofficial endpoint) for broader profile extraction is against LinkedIn's terms. The following two methods are both official, user-initiated, and carry no enforcement risk, since they use LinkedIn's own self-service export features rather than any form of automated access.

Method A (primary, recommended) -- "Save to PDF"

- LinkedIn has a built-in feature on every profile page (the "... More menu) to save that profile as a PDF -- this is an official, first-party export, available to any user for their own profile.
- In-product: show the user a short instruction card ("Open your LinkedIn profile -> click More -> Save to PDF") with a screenshot, then let them upload that PDF through the same upload flow as Section 1.
- No new parsing logic needed -- a LinkedIn-exported PDF is just a resume-shaped PDF to the same parser.
- This is the fastest to ship and the safest option, and should be the default path presented to users.

Method B (secondary, more complete) -- Official Data Export

- LinkedIn's Settings > "Get a copy of your data" produces a ZIP of CSV files (Profile.csv, Positions.csv, Education.csv, Skills.csv, and others) -- an official personal-data-portability feature.
- This gives cleaner, more structured data than PDF parsing (no layout-guessing needed at all -- it's already columnar), at the cost of more user friction: LinkedIn can take anywhere from a few minutes to up to 24 hours to generate the export, so this is better positioned as "for a more complete import, come back once your LinkedIn export is ready" rather than the default flow.
- Requires a small dedicated CSV parser (one mapping function per file: Positions.csv -> experience[], Education.csv -> education[], Skills.csv -> skills[]) -- straightforward, high-confidence mapping since the source is already structured.

Method C (optional, flag for legal review before building) -- extension-assisted read

- A browser-extension feature that reads the currently-open page when the user is logged into LinkedIn and viewing their own profile, on explicit click -- a single, user-initiated, client-side read of one page, not automated or repeated access.
- This is meaningfully different from the bulk scraping LinkedIn's enforcement targets (their own guidance ties detection to speed, repetition, and volume of access, not a single manual action) -- but it is still outside LinkedIn's officially sanctioned integration surface and carries some residual policy risk.
- Recommendation: do not build this for v1. If demand is high later, get a specific legal/ToS review before shipping it, and if built, gate it as strictly single-page, user-initiated, never automated or repeated in a session.

Explicitly out of scope, permanently: no server-side LinkedIn scraping, no bulk automation, and no third-party LinkedIn data-broker APIs. Beyond the ToS risk, building a core onboarding path on a data source that could be cut off or trigger legal action at any time is a foundation risk the product shouldn't take on.

4. The Review Screen (Applies to Every Import Source)

Regardless of source (resume upload, LinkedIn PDF, or LinkedIn data export), the user lands on the same review screen before anything is saved:

Field confidence	Review screen behavior
High-confidence fields	Pre-filled, shown normally, editable inline like any other Profile field.
Medium-confidence fields	Pre-filled but visually flagged (subtle highlight + small "please verify" note) -- still editable inline.
Low-confidence / unmatched text	Shown separately as "we found this but weren't sure where it goes" -- user can assign it to a field or discard it, never silently dropped or silently guessed into place.
Duplicate detection	If the user already has partial Profile data (e.g. re-importing after editing manually), matching fields are shown as "keep existing / use imported" choices rather than silently overwritten.
Skip/edit-later option	User can accept the import as-is and fix specific fields later from the normal Profile editor -- import is not a one-time, all-or-nothing gate.

5. File Handling & Privacy During Import

- The uploaded resume file itself is temporarily processed for extraction and then either discarded or stored as the user's canonical resume file (their choice on the review screen), consistent with the Profile Schema's `documents.resume.files` structure -- it is never retained as an unexplained duplicate.
- All the same encryption-in-transit and encryption-at-rest rules from the Autofill Engine Spec and Website Flow & Sync Spec apply to an uploaded resume file exactly as they apply to typed Profile data -- uploading is not a lower-trust code path that skips those protections.
- Google Drive Picker access is scoped to only the specific file the user selects (`drive.file` scope), never a broad `drive.readonly` grant -- the app should never be able to browse the user's Drive beyond what they explicitly picked.
- If parsing fails entirely (corrupted file, scanned-image-only PDF with no extractable text -- the same failure mode already handled in the ATS Checker), fail honestly: tell the user extraction didn't work and offer manual entry, rather than silently producing an empty or wrong Profile.

This entire import layer sits in front of the existing Profile Setup flow, not instead of it -- manual entry remains fully available and is the fallback whenever any import path fails or the user simply prefers to type.